Terraform build order — infrastructure as code, one file at a time

This converts everything you've already built by hand (VPC, EKS, RDS, IAM roles) into Terraform, in the order that actually works — each step only depends on steps before it, so you can `terraform apply` after each one and verify it before moving on. This is not "write it all, apply once" — build and verify incrementally, same discipline as your manual practice.

Goal state: `terraform apply` → full stack up. `terraform destroy` → everything gone, $0 billing. Same daily teardown habit you already have, just one command each way instead of dozens of manual steps.

Step 0 — Setup (do once)

- Install Terraform CLI (`brew install terraform` on Mac)
- Create a project folder, e.g. `terraform-microservices/`
- **State file decision:** for your use case (solo learner, destroying everything daily), a **local state file is genuinely fine** — don't add an S3 backend + DynamoDB locking yet. That complexity matters for *teams* working on the same infrastructure concurrently, which isn't you right now. Revisit this once you're collaborating with others or running something long-lived.

The build sequence

Each numbered item = roughly one `.tf` file, applied and checked before the next.

1. `00-providers.tf` — provider setup

Declares the AWS provider and region. Nothing gets created yet — this just tells Terraform *where* to create things.

2. `01-vpc.tf` — networking foundation

The VPC itself, all 6 subnets (public/private-app/private-data × 2 AZs), Internet Gateway, NAT Gateway, both route tables, and the required `kubernetes.io/role/elb` and `kubernetes.io/role/internal-elb` subnet tags. This is the largest single file and everything else depends on it — get this right and verified before moving on.

Depends on: nothing. First real resource to create.

3. `02-security-groups.tf` — firewalls

`Database-SG`, `Redis-SG` (and `Kafka-SG` once you add Kafka later). Reference the VPC and subnet CIDRs from step 2.

Depends on: the VPC (needs its ID and subnet CIDRs).

4. `03-ecr.tf` — image repositories

Your `myapp-backend` and `myapp-frontend` ECR repos — you've been creating these manually with `aws ecr create-repository`; this is the one-line Terraform equivalent.

Depends on: nothing (independent of the VPC).

5. `04-rds.tf` — database

RDS subnet group (using the private-data subnets from step 2), the PostgreSQL/MySQL instance itself, attached to `Database-SG`.

Depends on: VPC + subnets + Database-SG.

6. `05-elasticache.tf` — Redis

ElastiCache Redis cluster in the data subnets, attached to `Redis-SG`.

Depends on: VPC + subnets + Redis-SG.

7. `06-iam-eks.tf` — cluster IAM roles

The EKS cluster role (`AmazonEKSClusterPolicy`) and node group role (`AmazonEKSWorkerNodePolicy`, `AmazonEC2ContainerRegistryReadOnly`, `AmazonEKS_CNI_Policy`). Pure IAM, no dependency on the VPC yet.

Depends on: nothing.

8. `07-eks-cluster.tf` — the cluster itself

The EKS cluster and managed node group, using the roles from step 6 and the subnets from step 2. This is the step that takes ~15-20 minutes to apply — same as your manual `eksctl create cluster` runs, just declarative now.

Depends on: VPC + subnets + IAM roles.

9. `08-oidc-provider.tf` — cluster OIDC

The IAM OIDC provider tied to your cluster's issuer URL — the prerequisite for every IRSA role that follows (ALB controller, External Secrets). Terraform can read the cluster's OIDC issuer directly via a data source instead of you copy-pasting it from the console.

Depends on: the EKS cluster existing (needs its OIDC issuer URL).

10. `09-alb-controller-irsa.tf` — ALB controller permissions

The IAM policy + IRSA role for the AWS Load Balancer Controller, matching the trust policy you built manually earlier — now generated from the OIDC provider in step 9 instead of hand-edited JSON.

Depends on: OIDC provider.

11. `10-external-secrets-irsa.tf` — secrets access permissions

Same IRSA pattern, scoped role for the External Secrets Operator to read Secrets Manager.

Depends on: OIDC provider.

12. Secrets Manager — deliberately manual, not Terraform

Create the actual secret (`DB_USERNAME`, `DB_PASSWORD`, `REDIS_AUTH`) via console or CLI, same as before — **don't put secret values into Terraform**. Terraform state files store values in plain text by default, so committing real credentials into state defeats the purpose of using Secrets Manager at all. Terraform can reference the secret's **ARN**, never its contents.

13. `11-helm-alb-controller.tf` and `12-helm-argocd.tf` — the app-layer bootstrap

Using Terraform's `helm` provider, install the AWS Load Balancer Controller and ArgoCD directly as part of `terraform apply` — this is what makes your "one click, everything starts" goal real. Once ArgoCD is up, it takes over: watching your git repo and syncing your actual application manifests (frontend/backend Deployments, Services, Ingress) without any further Terraform or manual `kubectl apply` involved.

Depends on: the EKS cluster + OIDC/IRSA roles from steps 8-11.

What this gets you

```bash
terraform apply    # ~20 min: VPC → EKS → RDS/Redis → IRSA roles → ALB contro
# ArgoCD takes over from here, auto-deploying your app manifests from git
...work for the day...
terraform destroy   # everything gone, in reverse dependency order, $0 billin
```

Suggested pace

Don't write all 13 files in one sitting. One or two files a day, applying and verifying each before adding the next — same rhythm as the design patterns curriculum. Realistic order of days:

- **Day 1-2:** Steps 1-2 (VPC + security groups) — apply, confirm in the console, tear down, reapply to build confidence
- **Day 3:** Steps 3-4 (ECR + RDS)
- **Day 4:** Step 5 (Redis)
- **Day 5-6:** Steps 6-7 (IAM roles + the EKS cluster itself — the big one)
- **Day 7:** Steps 8-9 (OIDC + ALB controller IRSA)
- **Day 8:** Step 10 (External Secrets IRSA) + manual secret creation

- **Day 9-10:** Steps 11 (Helm provider — ALB controller + ArgoCD bootstrap) — the payoff step where it becomes genuinely one-click

Tell me when you're ready to start with `00-providers.tf` and `01-vpc.tf` and I'll write the actual Terraform code for those two first.